IMPLEMENTASI DAN PENGELOLAAN DOCKER SWARM SEBAGAI SISTEM CONTAINER ORCHESTRATION PADA VIRTUAL ENVIRONMENT

TUGAS WORKSHOP DEVELOPER OPERASIONAL

MINGGU KE 9

Oleh :

WILLY ADIMASTHA NUGROHO

E41230144

GOLONGAN A

PROGRAM STUDI TEKNIK INFORMATIKA JEMBER

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

2026

DAFTAR ISI

Halaman

DAFTAR ISI i
DAFTAR GAMBAR ii
BAB 1 PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
BAB 2 PEMBAHASAN 2
2.1 Acara 17 2
2.2 Acara 18 13
BAB 3 PENUTUP 16
3.1 Kesimpulan 16

DAFTAR GAMBAR

Halaman

Gambar 2.1 halaman utama multipass 2
Gambar 2.2 konfigurasi instance..... 2
Gambar 2.3 multipass stop & purge..... 3
Gambar 2.4 taskkill 3
Gambar 2.5 Instance manager dan worker berhasil dijalankan 4
Gambar 2.6 Akses shell ke instance manager 4
Gambar 2.7 Instalasi Docker engine pada instance manager melalui manajer paket APT 5
Gambar 2.8 Konfigurasi awal Docker Swarm pada node manager 5
Gambar 2.9 Integrasi worker node ke dalam cluster Docker Swarm 6
Gambar 2.10 Terminal instance manager menampilkan status sistem dan informasi jaringan 6
Gambar 2.11 Verifikasi status node manager dan worker pada Docker Swarm cluster 7
Gambar 2.12 Deployment dan verifikasi service web nginx pada Docker Swarm cluster 7
Gambar 2.13 Status infrastruktur virtual instance manager dan worker pada dashboard Multipass 8
Gambar 2.14 Verifikasi konfigurasi jaringan dan alamat IP pada instance manager 8
Gambar 2.15 Verifikasi konfigurasi jaringan dan identitas IP pada instance worker 9
Gambar 2.16 Verifikasi status Docker Swarm dan status firewall pada node manager 9
Gambar 2.17 Verifikasi konfigurasi jaringan, status Docker Swarm, dan firewall pada node worker 10
Gambar 2.18 Verifikasi status cluster Swarm dan daftar node aktif pada instance manager 11

Gambar 2.19 Validasi status worker node yang telah aktif dalam Docker Swarm cluster 11
Gambar 2.20 Verifikasi status Docker Swarm cluster dan daftar node aktif pada instance manager 12
Gambar 2.21 Verifikasi status aktif klaster dan topologi node manager serta worker 12
Gambar 2.22 Akses layanan nginx dari node manager melalui web browser 13
Gambar 2.23 Akses layanan nginx dari node worker melalui web browser..... 13
Gambar 2.24 Proses pembaruan image service web menjadi igdw/first-repo 14
Gambar 2.25 Verifikasi tampilan service yang telah diupdate dengan image igdw/first-repo pada kedua node 14
Gambar 2.26 Verifikasi tampilan service yang telah diupdate dengan image igdw/first-repo pada kedua node 14
Gambar 2.27 Detail konfigurasi dan status service web setelah pembaruan image 15
Gambar 2.28 riwayat task service web pada node manager dan worker 15
Gambar 2.29 Proses pelepasan instance worker dari klaster Docker Swarm 15

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Perkembangan arsitektur aplikasi modern semakin menuntut infrastruktur yang fleksibel, skalabel, dan memiliki ketahanan tinggi terhadap kegagalan sistem. Dalam lingkungan pengembangan dan operasional perangkat lunak, pengelolaan kontainer secara terisolasi tidak lagi cukup memadai ketika layanan harus menangani beban kerja yang dinamis serta kebutuhan ketersediaan yang berjalan terus-menerus. Oleh karena itu, pendekatan orkestrasi kontainer menjadi standar industri untuk memastikan distribusi sumber daya yang efisien, pemeliharaan layanan yang terpusat, serta pemulihan otomatis saat terjadi gangguan.

Docker Swarm hadir sebagai solusi bawaan yang memungkinkan beberapa host Docker digabungkan menjadi satu klaster terpadu, sehingga beroperasi layaknya satu server virtual tunggal. Arsitektur ini membagi peran secara jelas antara node pengelola yang bertanggung jawab atas koordinasi dan pengambilan keputusan klaster, serta node pekerja yang menjalankan eksekusi tugas sesuai instruksi.

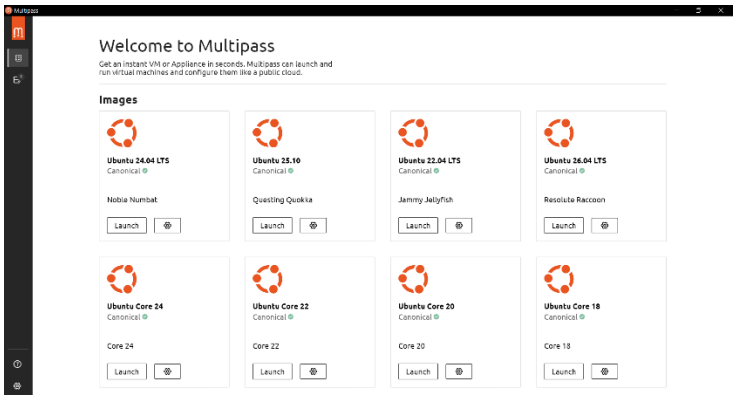
1.2 Tujuan

1. Memahami prinsip dasar dan konsep teoritis Docker Swarm sebagai alat orkestrasi kontainer dalam membangun klaster terdistribusi.
2. Menguasai prosedur inisialisasi klaster, pengelolaan token keamanan, serta konfigurasi jaringan dan firewall untuk memastikan komunikasi antar node berjalan optimal.
3. Melaksanakan pengelolaan layanan secara menyeluruh, mulai dari pembuatan, pemantauan, pembaruan image, penskalaan replika, hingga prosedur aman saat node keluar dari klaster.

BAB 2 PEMBAHASAN

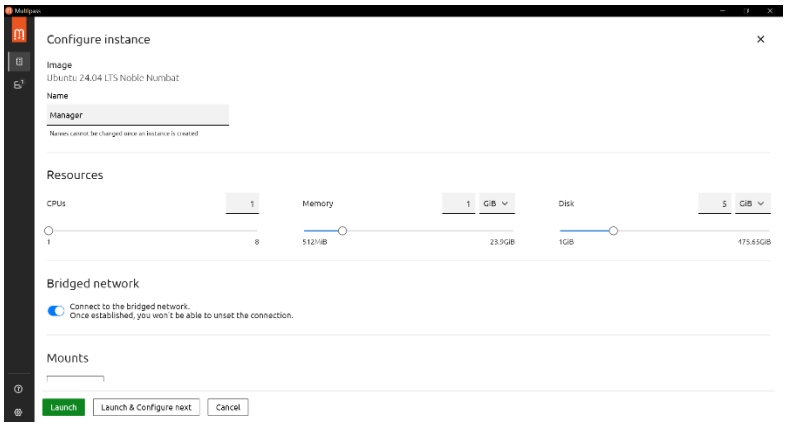
2.1 Acara 17

Tahapan awal memperlihatkan proses pembuatan instance virtual yang akan difungsikan sebagai manager. Konfigurasi sumber daya seperti CPU, memori, dan penyimpanan disesuaikan dengan kebutuhan minimal agar sistem tetap ringan namun mampu menjalankan Docker secara optimal.



Gambar 2.1 halaman utama multipass

Langkah berikutnya menunjukkan pembuatan instance yang berperan sebagai manager dan worker. Penggunaan spesifikasi yang sama bertujuan menjaga keseimbangan performa antar node sehingga distribusi tugas dapat berjalan dengan stabil.



Gambar 2.2 konfigurasi instance

Upaya pembersihan dilanjutkan dengan menghentikan semua instance dan melakukan purge ulang, tetapi menghadapi kendala ketika mencoba menghentikan layanan Multipass yang berulang kali menunjukkan pesan bahwa layanan sedang dalam proses starting atau stopping sehingga perlu menunggu hingga operasi tersebut selesai.

```
PS C:\Windows\System32> multipass stop --all
PS C:\Windows\System32> multipass delete --all
PS C:\Windows\System32> multipass purge
PS C:\Windows\System32> net stop multipass
The Multipass Service service is stopping.
The Multipass Service service was stopped successfully.

PS C:\Windows\System32> net start multipass
The Multipass Service service is starting.
The Multipass Service service was started successfully.

PS C:\Windows\System32> multipass launch --name manager
launch failed: Hash of C:/ProgramData/Multipass/cache/vault/images/noble-20260321/ubuntu-24.04-server-cloudimg-amd64.img
  does not match 5c3ddb00f60bc455dac0862fabe9d8bacec46c33ac1751143c5c3683404b110d
PS C:\Windows\System32> multipass stop --all
PS C:\Windows\System32> multipass delete --all
PS C:\Windows\System32> multipass purge
PS C:\Windows\System32> net stop multipass
The Multipass Service service is stopping.......
The Multipass Service service could not be stopped.

PS C:\Windows\System32> net stop multipass
The service is starting or stopping.  Please try again later.

PS C:\Windows\System32> net stop multipass
The service is starting or stopping.  Please try again later.

PS C:\Windows\System32> net stop multipass
The service is starting or stopping.  Please try again later.
```

Gambar 2.3 multipass stop & purge

Pesan kesalahan menunjukkan kegagalan pengunduh URL dalam mengakses file ubuntu-22.04-server-cloudimg-amd64.img, yang kemudian diperparah dengan ketidakcocokan hash kriptografis yang mengindikasikan file cache yang tersimpan mengalami kerusakan atau tidak lengkap. Upaya pembersihan dilanjutkan dengan menghentikan semua instance, menghapus konfigurasi, dan melakukan purge total sebelum akhirnya berhasil menghentikan layanan Multipass secara sempurna.

```
PS C:\Windows\System32> taskkill /F /IM multipassd.exe
SUCCESS: The process "multipassd.exe" with PID 13968 has been terminated.
PS C:\Windows\System32> taskkill /F /IM multipass.exe
ERROR: The process "multipass.exe" not found.
PS C:\Windows\System32> net start multipass
The requested service has already been started.

More help is available by typing NET HELPMSG 2182.

PS C:\Windows\System32> multipass list
No instances found.
PS C:\Windows\System32> multipass delete --all
PS C:\Windows\System32> multipass purge
PS C:\Windows\System32> multipass launch 22.04 --name manager
[2026-04-15T09:20:38.403] [error] [url downloader] Failed to get https://cloud-images.ubuntu.com/releases/server/release
s/jammy/release-20260320/ubuntu-22.04-server-cloudimg-amd64.img: Error opening https://cloud-images.ubuntu.com/releases/
server/releases/jammy/release-20260320/ubuntu-22.04-server-cloudimg-amd64.img
launch failed: failed to download from 'https://cloud-images.ubuntu.com/releases/server/releases/jammy/release-20260320/
ubuntu-22.04-server-cloudimg-amd64.img': Error opening https://cloud-images.ubuntu.com/releases/server/releases/jammy/re
lease-20260320/ubuntu-22.04-server-cloudimg-amd64.img
PS C:\Windows\System32> multipass launch 22.04 --name manager
launch failed: Hash of C:/ProgramData/Multipass/cache/vault/images/jammy-20260320/ubuntu-22.04-server-cloudimg-amd64.img
  does not match ea85b16f81b3f6aa53a1260912d3f991fc33e0e0fc1d73f0b8c9c96247e42fdb
PS C:\Windows\System32> multipass stop --all
PS C:\Windows\System32> multipass delete --all
PS C:\Windows\System32> multipass purge
PS C:\Windows\System32> net stop multipass
The Multipass Service service is stopping.
The Multipass Service service was stopped successfully.
```

Gambar 2.4 taskkill

Setelah melakukan pembersihan total melalui penghentian layanan, penghapusan instance, dan purge cache, pengguna memutuskan beralih ke versi Ubuntu 20.04 LTS Focal Fossa yang lebih stabil. Perintah multipass launch focal --name manager berhasil mengeksekusi peluncuran instance pertama yang berfungsi sebagai node manajer, diikuti dengan perintah serupa untuk instance worker.

```
[2026-04-15T09:20:38.403] [error] [url downloader] Failed to get https://cloud-images.ubuntu.com/releases/server/release
s/jammy/release-20260320/ubuntu-22.04-server-cloudimg-amd64.img: Error opening https://cloud-images.ubuntu.com/releases/
server/releases/jammy/release-20260320/ubuntu-22.04-server-cloudimg-amd64.img
launch failed: failed to download from 'https://cloud-images.ubuntu.com/releases/server/releases/jammy/release-20260320/
ubuntu-22.04-server-cloudimg-amd64.img': Error opening https://cloud-images.ubuntu.com/releases/server/releases/jammy/re
lease-20260320/ubuntu-22.04-server-cloudimg-amd64.img
PS C:\Windows\System32> multipass launch 22.04 --name manager
launch failed: Hash of C:/ProgramData/Multipass/cache/vault/images/jammy-20260320/ubuntu-22.04-server-cloudimg-amd64.img
 does not match ea85b16f81b3f6aa53a1260912d3f991fc33e0e0fc1d73f0b8c9c96247e42fdb
PS C:\Windows\System32> multipass stop --all
PS C:\Windows\System32> multipass delete --all
PS C:\Windows\System32> multipass purge
PS C:\Windows\System32> net stop multipass
The Multipass Service service is stopping.
The Multipass Service service was stopped successfully.

PS C:\Windows\System32> net start multipass
The Multipass Service service is starting.
The Multipass Service service was started successfully.

PS C:\Windows\System32> multipass launch focal --name manager
Launched: manager
PS C:\Windows\System32> multipass launch focal --name worker
Launched: worker
PS C:\Windows\System32> multipass list
Name                    State             IPv4             Image
manager                 Running           172.27.198.142   Ubuntu 20.04 LTS
worker                  Running           172.27.199.51    Ubuntu 20.04 LTS
```

Gambar 2.5 Instance manager dan worker berhasil dijalankan

Alamat IPv4 172.27.198.142 yang tertera pada antarmuka eth0 sesuai dengan hasil pemeriksaan sebelumnya melalui perintah multipass list, memvalidasi bahwa konfigurasi jaringan telah berfungsi dengan baik.

```
PS C:\Windows\System32> multipass shell manager
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-216-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Wed Apr 15 11:34:59 WIB 2026

  System load:  0.0               Processes:             104
  Usage of /:   35.0% of 4.67GB   Users logged in:       0
  Memory usage: 23%               IPv4 address for eth0: 172.27.198.142
  Swap usage:   0%


Expanded Security Maintenance for Infrastructure is not enabled.

21 updates can be applied immediately.
15 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Infra to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '22.04.5 LTS' available.
Run 'do-release-upgrade' to upgrade to it.
```

Gambar 2.6 Akses shell ke instance manager

Docker telah berhasil diintegrasikan ke dalam sistem operasi host sebagai fondasi teknis sebelum dilanjutkan ke fase inisialisasi cluster Swarm.

```
ubuntu@manager:~$ sudo apt update
Hit:1 http://archive.ubuntu.com/ubuntu focal InRelease
Hit:2 http://archive.ubuntu.com/ubuntu focal-updates InRelease
Hit:3 http://archive.ubuntu.com/ubuntu focal-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu focal-security InRelease
Reading package lists... Done
Building dependency tree
Reading state information... Done
21 packages can be upgraded. Run 'apt list --upgradable' to see them.
ubuntu@manager:~$ sudo apt install docker.io -y
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  bridge-utils containerd dns-root-data dnsmasq-base libidn11 pigz runc ubuntu-fan
Suggested packages:
  ifupdown aufs-tools cgroupfs-mount | cgroup-lite debootstrap docker-buildx docker-compose-v2 docker-doc rinse
  zfs-fuse | zfsutils
The following NEW packages will be installed:
  bridge-utils containerd dns-root-data dnsmasq-base docker.io libidn11 pigz runc ubuntu-fan
0 upgraded, 9 newly installed, 0 to remove and 21 not upgraded.
Need to get 71.6 MB of archives.
After this operation, 301 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu focal/universe amd64 pigz amd64 2.4-1 [57.4 kB]
Get:2 http://archive.ubuntu.com/ubuntu focal/main amd64 bridge-utils amd64 1.6-2ubuntu1 [30.5 kB]
Get:3 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 runc amd64 1.1.12-0ubuntu2~20.04.1 [8066 kB]
Get:4 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 containerd amd64 1.7.24-0ubuntu1~20.04.2 [33.6 MB]
Get:5 http://archive.ubuntu.com/ubuntu focal-updates/main amd64 dns-root-data all 2024071801~ubuntu0.20.04.1 [6128 B]
```

Gambar 2.7 Instalasi Docker engine pada instance manager melalui manajer paket APT

Perintah ip a mengonfigurasi antarmuka jaringan eth0 dengan alamat IPv4 172.27.198.142 yang akan digunakan sebagai acuan dalam inisialisasi cluster. Proses inisialisasi Docker Swarm mengalami kendala pada percobaan pertama akibat kesalahan sintaksis penulisan perintah, kemudian dilanjutkan dengan pesan "permission denied" yang mengindikasikan kebutuhan hak akses di dalamnya.

```
Created symlink /etc/systemd/system/sockets.target.wants/docker.socket → /lib/systemd/system/docker.socket.
Setting up dnsmasq-base (2.90-0ubuntu0.20.04.1) ...
Setting up ubuntu-fan (0.12.13ubuntu0.1) ...
Created symlink /etc/systemd/system/multi-user.target.wants/ubuntu-fan.service → /lib/systemd/system/ubuntu-fan.service.
Processing triggers for systemd (245.4-4ubuntu3.24) ...
Processing triggers for man-db (2.9.1-1) ...
ubuntu@manager:~$ sudo systemctl start docker
ubuntu@manager:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 52:54:00:92:a2:f1 brd ff:ff:ff:ff:ff:ff
    inet 172.27.198.142/20 metric 100 brd 172.27.207.255 scope global dynamic eth0
       valid_lft 82562sec preferred_lft 82562sec
    inet6 fe80::5054:ff:fe92:a2f1/64 scope link
       valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:ef:6d:ad:ac brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
ubuntu@manager:~$ docker swarm init --advertise-addr <IP_MANAGER>
-bash: syntax error near unexpected token `newline'
ubuntu@manager:~$ docker swarm init --advertise-addr 172.27.198.142
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Post "http://%2Fva
r%2Frun%2Fdocker.sock/v1.45/swarm/init": dial unix /var/run/docker.sock: connect: permission denied
ubuntu@manager:~$ sudo docker swarm init --advertise-addr 172.27.198.142
```

Gambar 2.8 Konfigurasi awal Docker Swarm pada node manager

Setelah Docker versi 26.1.3 terinstal dan diverifikasi, instance worker bergabung dengan cluster Swarm menggunakan token autentikasi dan alamat IP manager 172.27.198.142 pada port 2377. Pesan "This node joined a swarm as a worker"

mengonfirmasi bahwa integrasi node worker ke dalam arsitektur cluster telah berhasil dilakukan.

```
Setting up docker.io (26.1.3-0ubuntu1~20.04.1) ...
Adding group `docker' (GID 120) ...
Done.
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /lib/systemd/system/docker.service.
Created symlink /etc/systemd/system/sockets.target.wants/docker.socket → /lib/systemd/system/docker.socket.
Setting up dnsmasq-base (2.90-0ubuntu0.20.04.1) ...
Setting up ubuntu-fan (0.12.13ubuntu0.1) ...
Created symlink /etc/systemd/system/multi-user.target.wants/ubuntu-fan.service → /lib/systemd/system/ubuntu-fan.service.
Processing triggers for systemd (245.4-4ubuntu3.24) ...
Processing triggers for man-db (2.9.1-1) ...
Processing triggers for dbus (1.12.16-2ubuntu2.3) ...
Processing triggers for libc-bin (2.31-0ubuntu9.18) ...
ubuntu@worker:~$ sudo systemctl start docker
ubuntu@worker:~$ docker --version
Docker version 26.1.3, build 26.1.3-0ubuntu1~20.04.1
ubuntu@worker:~$ sudo docker swarm join --token SWMTKN-1-1y2o6x614gztz1cm0z3geaqo2sgg66egw5xcy975e62hwv1hwa-agl0nzku1sad
8f866zx8v8dn2 172.27.198.142:2377
This node joined a swarm as a worker.
ubuntu@worker:~$ multipass shell manager
multipass: command not found
ubuntu@worker:~$
```

Gambar 2.9 Integrasi worker node ke dalam cluster Docker Swarm

Instance manager berada dalam status aktif dan siap untuk dilakukan konfigurasi lanjutan dalam pengelolaan Docker Swarm cluster.

```
S C:\Windows\System32> ubuntu@worker:~$ multipass shell manager
buntu@worker:~$: The term 'ubuntu@worker:~$' is not recognized as a name of a cmdlet, function, script file, or executa
ble program.
heck the spelling of the name, or if a path was included, verify that the path is correct and try again.
S C:\Windows\System32> multipass shell manager
elcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.4.0-216-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Wed Apr 15 12:31:30 WIB 2026

  System load:  0.27              Processes:             106
  Usage of /:   43.8% of 4.67GB   Users logged in:       0
  Memory usage: 30%               IPv4 address for eth0: 172.27.198.142
  Swap usage:   0%

 * Ubuntu 20.04 LTS Focal Fossa has reached its end of standard support
   on 31 May 2025.

   For more details see:
   https://ubuntu.com/20-04

xpanded Security Maintenance for Infrastructure is not enabled.

1 updates can be applied immediately.
5 of these updates are standard security updates.
o see these additional updates run: apt list --upgradable
```

Gambar 2.10 Terminal instance manager menampilkan status sistem dan informasi jaringan

Percobaan awal mengalami penolakan akses saat mencoba berkomunikasi dengan soket daemon Docker, sehingga memerlukan elevasi hak istimewa menggunakan prefiks sudo agar perintah dapat dieksekusi secara penuh. Setelah penyesuaian hak akses, keluaran terminal mengonfirmasi bahwa topologi klaster telah berhasil dibentuk dengan dua simpul yang aktif: satu node manager yang bertindak sebagai pemimpin klaster dan satu node worker. Kedua simpul menunjukkan status siap serta ketersediaan aktif dengan versi mesin Docker yang identik, menandakan bahwa proses inisialisasi dan penggabungan node telah berjalan sesuai prosedur.

```
ubuntu@manager: ~

* Ubuntu 20.04 LTS Focal Fossa has reached its end of standard support
  on 31 May 2025.

  For more details see:
  https://ubuntu.com/20-04

Expanded Security Maintenance for Infrastructure is not enabled.

21 updates can be applied immediately.
15 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

101 additional security updates can be applied with ESM Infra.
Learn more about enabling ESM Infra service for Ubuntu 20.04 at
https://ubuntu.com/20-04

New release '22.04.5 LTS' available.
Run 'do-release-upgrade' to upgrade to it.


Last login: Wed Apr 15 11:35:00 2026 from 172.27.192.1
ubuntu@manager:~$ docker node ls
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Get "http://%2Fvar
%2Frun%2Fdocker.sock/v1.45/nodes": dial unix /var/run/docker.sock: connect: permission denied
ubuntu@manager:~$ sudo docker node ls
ID                            HOSTNAME   STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
v1i211g7hr9nxv2vseclj7pl3 *   manager    Ready     Active         Leader           26.1.3
kbnkwjhuslb963f8afic20rq7     worker     Ready     Active                          26.1.3
ubuntu@manager:~$
```

Gambar 2.11 Verifikasi status node manager dan worker pada Docker Swarm cluster

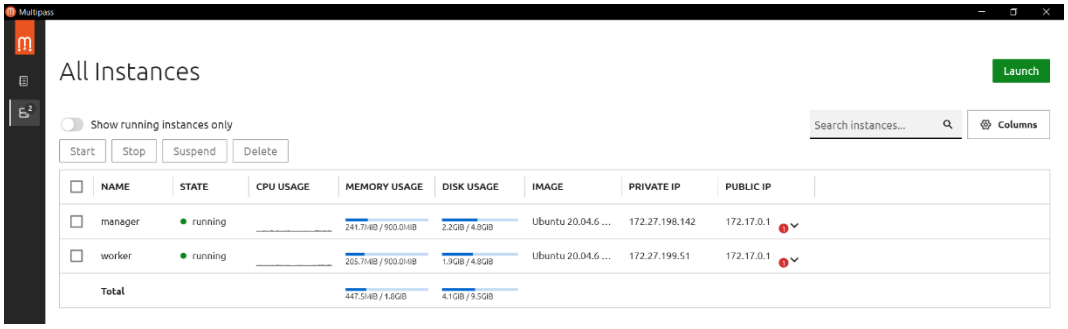
Proses deployment layanan web ke dalam klaster Docker Swarm yang telah terbentuk. Setelah verifikasi daftar node menunjukkan satu manager dan satu worker dalam status siap, perintah docker service create --name web -p 80:80 nginx dieksekusi untuk membuat layanan berbasis image nginx dengan pemetaan port 80. Progres deployment menunjukkan bahwa task berhasil dikonvergensi dan berjalan pada node manager. Pemeriksaan melalui docker service ls mengonfirmasi bahwa service web beroperasi dalam mode replicated dengan satu replika aktif, sedangkan perintah docker service ps web memberikan detail eksekusi task yang menunjukkan status running selama dua menit tanpa error.

```
New release '22.04.5 LTS' available.
Run 'do-release-upgrade' to upgrade to it.


Last login: Wed Apr 15 11:35:00 2026 from 172.27.192.1
ubuntu@manager:~$ docker node ls
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Get "http://%2Fvar
%2Frun%2Fdocker.sock/v1.45/nodes": dial unix /var/run/docker.sock: connect: permission denied
ubuntu@manager:~$ sudo docker node ls
ID                            HOSTNAME   STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
v1i211g7hr9nxv2vseclj7pl3 *   manager    Ready     Active         Leader           26.1.3
kbnkwjhuslb963f8afic20rq7     worker     Ready     Active                          26.1.3
ubuntu@manager:~$ sudo docker service create --name web -p 80:80 nginx
gw8out7epbdvw4mz899mou6g3
overall progress: 1 out of 1 tasks
1/1: running   [==================================================>]
verify: Service gw8out7epbdvw4mz899mou6g3 converged
ubuntu@manager:~$ sudo docker service ls
ID             NAME      MODE         REPLICAS   IMAGE          PORTS
gw8out7epbdv   web       replicated   1/1        nginx:latest   *:80->80/tcp
ubuntu@manager:~$ sudo docker service ps web
ID             NAME      IMAGE          NODE      DESIRED STATE   CURRENT STATE           ERROR     PORTS
mghn530klr66   web.1     nginx:latest   manager   Running         Running 2 minutes ago
ubuntu@manager:~$
```

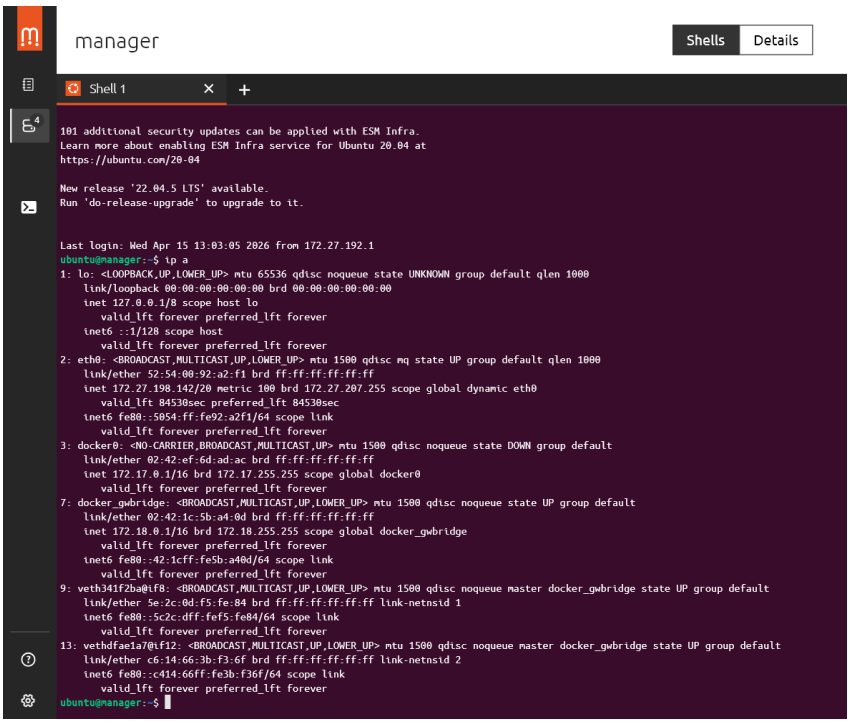
Gambar 2.12 Deployment dan verifikasi service web nginx pada Docker Swarm cluster

Dua instance bernama manager dan worker tercatat dalam status *running*, mengonfirmasi bahwa kedua node telah beroperasi.



Gambar 2.13 Status infrastruktur virtual instance manager dan worker pada dashboard Multipass

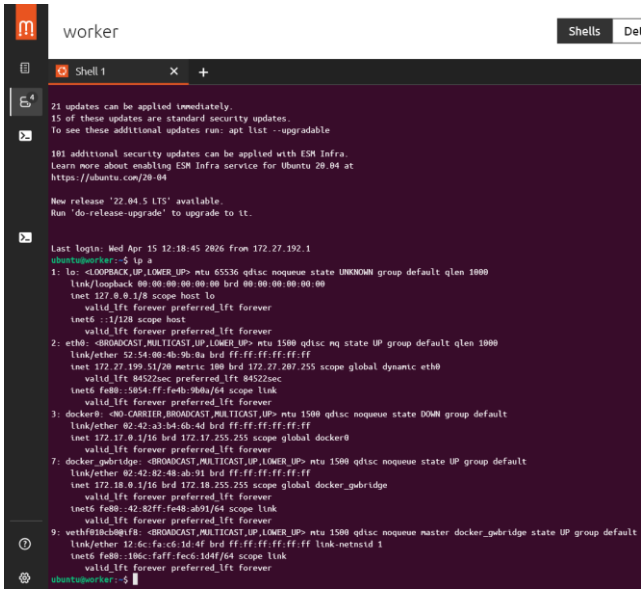
Antarmuka eth0 mengonfirmasi bahwa node manager telah terpasang dengan benar pada jaringan virtual Multipass dengan alamat IPv4 172.27.198.142. Keberadaan antarmuka docker0 dan docker_gwbridge menunjukkan bahwa lapisan jaringan Docker telah terinisialisasi.



Gambar 2.14 Verifikasi konfigurasi jaringan dan alamat IP pada instance manager

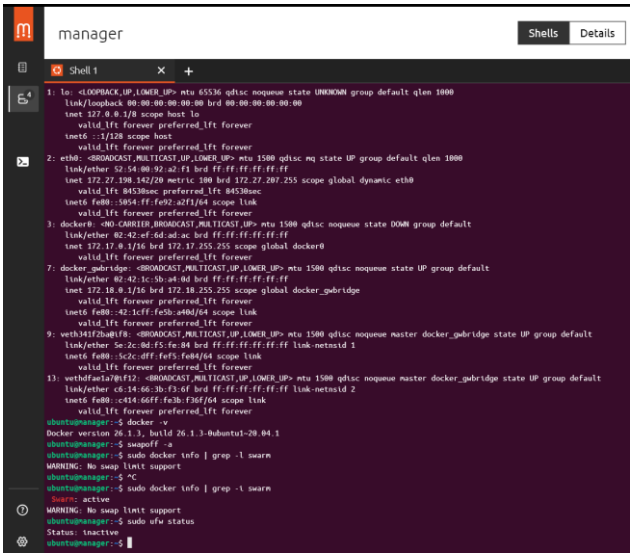
node worker telah terhubung ke jaringan virtual dengan alamat IPv4 172.27.199.51, yang berfungsi sebagai identitas unik dalam klaster. Keberadaan antarmuka

docker_gwbridge serta veth (virtual ethernet) menunjukkan bahwa lapisan jaringan overlay Docker Swarm telah terbentuk di sisi worker.



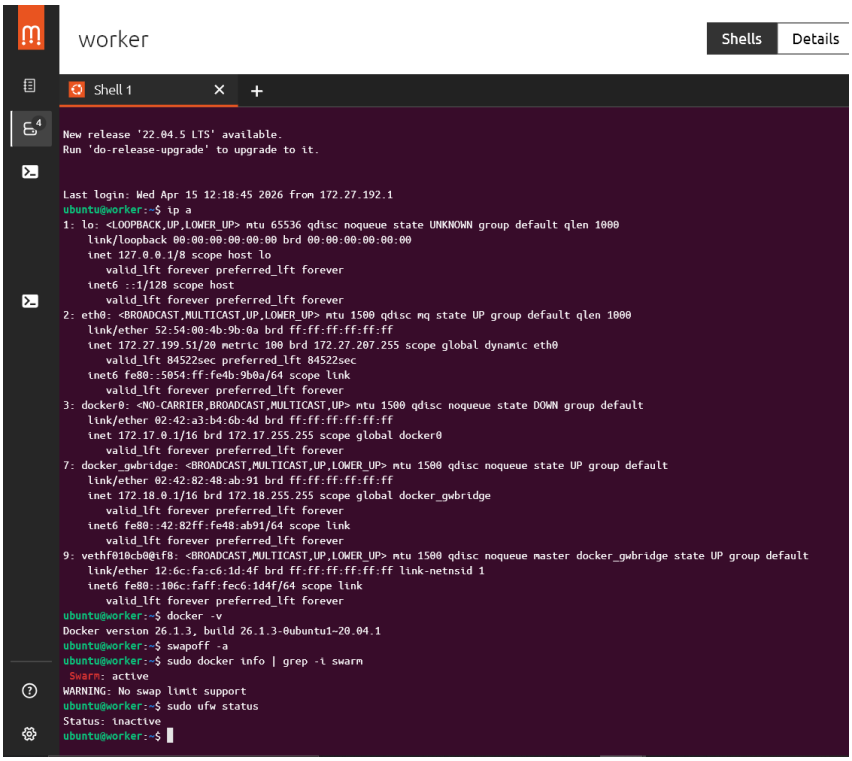
Gambar 2.15 Verifikasi konfigurasi jaringan dan identitas IP pada instance worker

Administrator kemudian menonaktifkan memori swap menggunakan swapoff -a sebagai langkah optimalisasi performa kontainer. Pengecekan status menggunakan sudo docker info | grep -i swarm memvalidasi bahwa mode Docker Swarm telah active. Terakhir, perintah sudo ufw status mengonfirmasi bahwa firewall berada dalam kondisi inactive, memastikan bahwa port komunikasi krusial untuk klaster tidak terblokir dan konektivitas antar node dapat berjalan lancar.



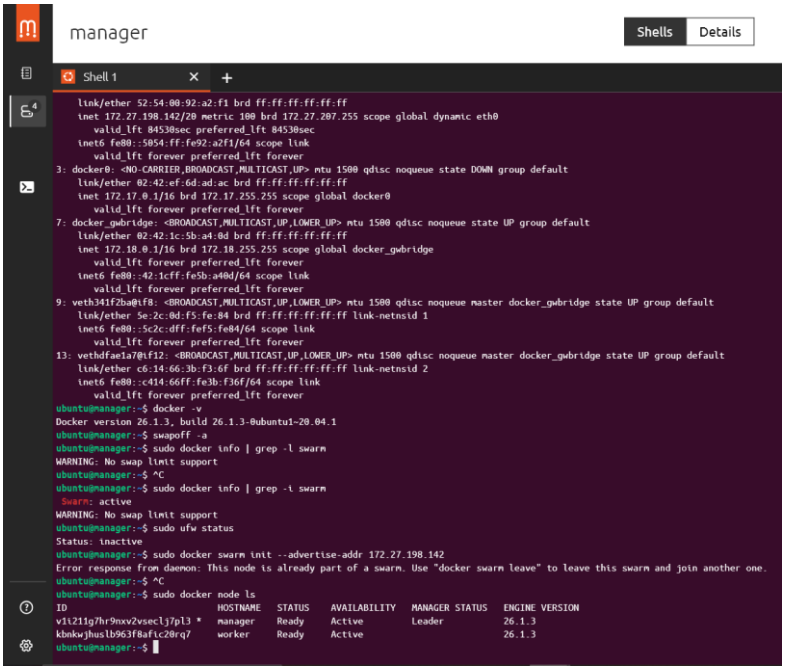
Gambar 2.16 Verifikasi status Docker Swarm dan status firewall pada node manager

Penonaktifan swap melalui perintah swapoff -a dilakukan sebagai langkah optimalisasi kinerja kontainer. Verifikasi status Docker Swarm menunjukkan hasil "Swarm: active", mengonfirmasi bahwa node worker telah berhasil bergabung dengan klaster. Pengecekan firewall mengungkapkan status "inactive", memastikan tidak ada hambatan pada port komunikasi yang diperlukan untuk interaksi antar node dalam klaster Docker Swarm.



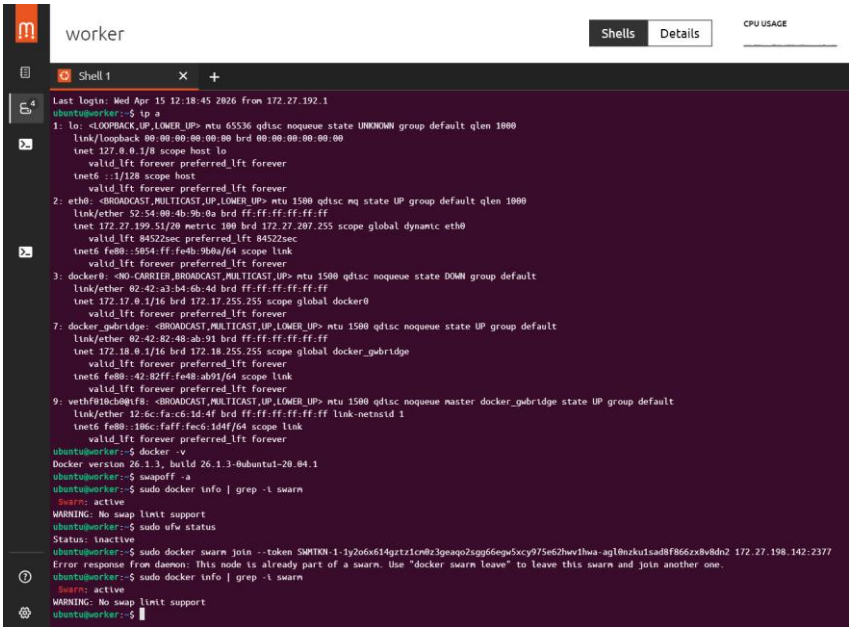
Gambar 2.17 Verifikasi konfigurasi jaringan, status Docker Swarm, dan firewall pada node worker

Setelah menonaktifkan swap dan memeriksa versi Docker, status Swarm dikonfirmasi aktif melalui perintah docker info. Pemeriksaan firewall (ufw status) menunjukkan status nonaktif, memastikan komunikasi antar node tidak terhalang. Percobaan inisialisasi ulang swarm gagal karena node sudah tergabung dalam cluster. Perintah docker node ls berhasil menampilkan topologi cluster yang terdiri dari node manager sebagai Leader dan node worker yang berstatus Ready dan Active, menandakan infrastruktur Swarm telah siap digunakan.



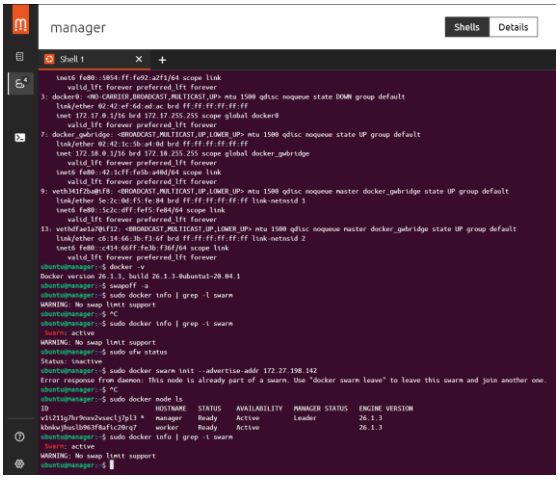
Gambar 2.18 Verifikasi status cluster Swarm dan daftar node aktif pada instance manager

Setelah pemeriksaan antarmuka jaringan dan penonaktifan memori swap, status Docker Swarm dikonfirmasi dalam keadaan *active*. Ketika pengguna mengeksekusi perintah sudo docker swarm join dengan token dan IP manager 172.27.198.142, sistem menolak proses tersebut dengan pesan bahwa node ini sudah merupakan bagian dari sebuah swarm.



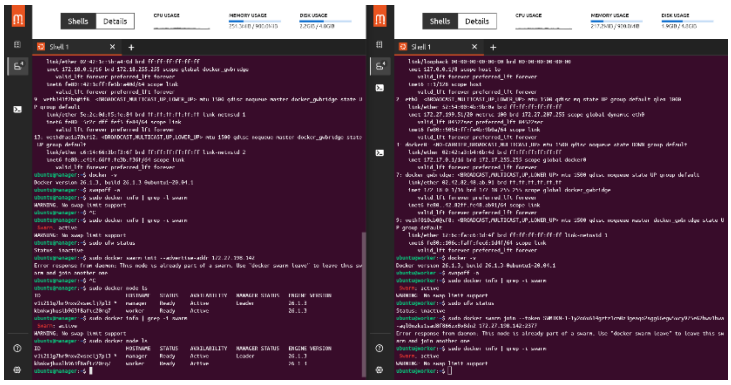
Gambar 2.19 Validasi status worker node yang telah aktif dalam Docker Swarm cluster

Pengecekan status swarm mengungkapkan bahwa Docker Swarm telah dalam kondisi *active*, sedangkan firewall UFW berada dalam status *inactive* untuk memastikan komunikasi antar node tidak terblokir. Percobaan inisialisasi ulang swarm ditolak oleh sistem karena node telah tergabung dalam cluster, yang justru memvalidasi bahwa konfigurasi sebelumnya berhasil. Perintah docker node ls berhasil menampilkan topologi cluster dengan node manager berstatus *Leader* dan node worker berstatus *Ready* dan *Active.*



Gambar 2.20 Verifikasi status Docker Swarm cluster dan daftar node aktif pada instance manager

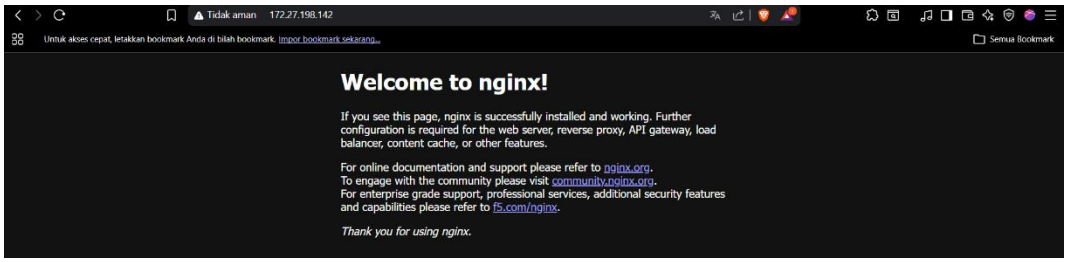
Percobaan inisialisasi ulang pada manager dan penggabungan ulang pada worker secara sengaja menghasilkan penolakan dari daemon, yang justru memvalidasi bahwa kedua node telah terintegrasi dengan sukses dalam arsitektur klaster. Perintah docker node ls pada sisi manager berhasil menampilkan topologi lengkap, dengan node manager bertindak sebagai pemimpin (*Leader*) dan node worker berstatus siap (*Ready*) serta aktif (*Active*).



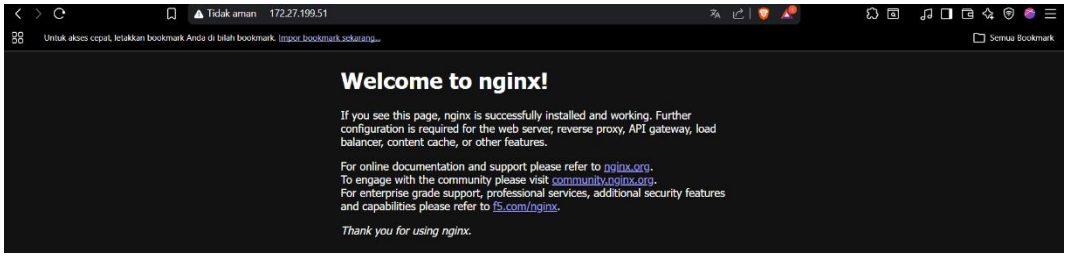
Gambar 2.21 Verifikasi status aktif klaster dan topologi node manager serta worker

2.2 Acara 18

Web berhasil mengakses layanan nginx yang telah di-deploy pada klaster Docker Swarm melalui dua alamat IP berbeda. Akses pertama dilakukan melalui alamat IP manager 172.27.198.142, sedangkan akses kedua melalui alamat IP worker 172.27.199.51. Kedua permintaan berhasil dilayani dan menampilkan halaman selamat datang nginx yang identik, mengonfirmasi bahwa mekanisme routing internal dan load balancing pada Docker Swarm berfungsi dengan baik.



Gambar 2.22 Akses layanan nginx dari node manager melalui web browser



Gambar 2.23 Akses layanan nginx dari node worker melalui web browser

Menampilkan eksekusi perintah pembaruan layanan *web* melalui sudo docker service update --image igdw/first-repo web. Tampilan ini mendemonstrasikan fleksibilitas Docker Swarm dalam mengganti image aplikasi secara dinamis tanpa perlu menghentikan layanan secara manual. Pesan "overall progress: 2 out of 2 tasks" serta status "verify: Service web converged" mengindikasikan bahwa semua task telah diperbarui dengan sukses menggunakan image igdw/first-repo, dan kondisi layanan telah stabil kembali.

```
ubuntu@manager:~$ sudo docker service update --image igdw/first-repo web
web
overall progress: 2 out of 2 tasks
1/2: running   [==================================================>]
2/2: running   [==================================================>]
verify: Service web converged
```

Gambar 2.24 Proses pembaruan image service web menjadi igdw/first-repo

Web berhasil menampilkan konten dari layanan yang telah diperbarui menggunakan image igdw/first-repo melalui dua alamat IP berbeda dalam klaster Docker Swarm. Akses pertama dilakukan melalui alamat 172.23.60.54 dan akses kedua melalui 172.23.51.108

Gambar 2.25 Verifikasi tampilan service yang telah diupdate dengan image igdw/first-repo pada kedua node

Gambar 2.26 Verifikasi tampilan service yang telah diupdate dengan image igdw/first-repo pada kedua node

Keluaran ini memberikan gambaran terstruktur mengenai konfigurasi layanan, mulai dari mode replikasi dengan dua salinan (*replicas: 2*), status pembaruan yang telah berstatus *completed*, hingga spesifikasi kontainer yang kini secara aktif menggunakan image igdw/first-repo:latest. Informasi jaringan juga terpampang jelas, di mana layanan dipublikasikan pada port 80 dengan protokol TCP melalui mode *ingress*, yang memungkinkan akses lintas node dalam klaster. Bagian konfigurasi pembaruan dan pengembalian (*UpdateConfig* dan *RollbackConfig*) menguraikan strategi yang diterapkan, seperti paralelisme satu per satu, penangguhan saat gagal, serta urutan penghentian *stop-first*. Hasil inspeksi ini mengonfirmasi bahwa layanan telah berhasil diperbarui sesuai instruksi, berjalan stabil, dan siap menerima lalu lintas, sekaligus menjadi referensi teknis penting dalam proses audit dan manajemen orkestrasi kontainer.

```
ubuntu@manager:~$ sudo docker service inspect web --pretty

ID:             uhr3n3mdhufi30j5ic4zyn33x
Name:           web
Service Mode:   Replicated
 Replicas:      2
UpdateStatus:
 State:         completed
 Started:       8 minutes ago
 Completed:     3 minutes ago
 Message:       update completed
Placement:
UpdateConfig:
 Parallelism:   1
 On failure:    pause
 Monitoring Period: 5s
 Max failure ratio: 0
 Update order:      stop-first
RollbackConfig:
 Parallelism:   1
 On failure:    pause
 Monitoring Period: 5s
 Max failure ratio: 0
 Rollback order:    stop-first
ContainerSpec:
 Image:         igdw/first-repo:latest@sha256:8bf3db64f02fe172d48dd07e5aee32ea7eea6f7976eaa2a752a087811b63652b
 Init:          false
Resources:
Endpoint Mode:  vip
Ports:
 PublishedPort = 80
  Protocol = tcp
  TargetPort = 80
  PublishMode = ingress
```

Gambar 2.27 Detail konfigurasi dan status service web setelah pembaruan image

Keluaran ini mengungkap riwayat dan status terkini dari setiap replika service, di mana task yang menggunakan image lama nginx:latest telah berstatus Shutdown setelah digantikan oleh task baru berbasis igdw/first-repo:latest yang berstatus Running. Distribusi task terlihat merata pada node manager dan worker.

```
ubuntu@manager:~$ sudo docker service ps web
ID             NAME        IMAGE                  NODE      DESIRED STATE   CURRENT STATE            ERROR     PORTS
61k5399mg14y   web.1       igdw/first-repo:latest manager   Running         Running 5 minutes ago
nicmz22dbbp7    \_ web.1   nginx:latest           manager   Shutdown        Shutdown 5 minutes ago
j1kji528jocw   web.2       igdw/first-repo:latest worker    Running         Running 5 minutes ago
4pe68xb61mgn    \_ web.2   nginx:latest           worker    Shutdown        Shutdown 10 minutes ago
```

Gambar 2.28 riwayat task service web pada node manager dan worker

Melalui eksekusi perintah sudo docker swarm leave, node worker secara aktif memutus kaitan dengan manager. Keluaran "Node left the swarm" mengonfirmasi bahwa proses deprovisioning telah berhasil, menandakan bahwa instance tersebut kini berdiri mandiri dan tidak lagi berpartisipasi dalam orkestrasi atau penjadwalan task di dalam klaster.

```
ubuntu@worker:~$ sudo docker swarm leave
Node left the swarm.
ubuntu@worker:~$
```

Gambar 2.29 Proses pelepasan instance worker dari klaster Docker Swarm

BAB 3 PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa Docker Swarm merupakan solusi orkestrasi kontainer yang efektif dalam membangun sistem terdistribusi dengan konsep cluster yang terintegrasi. Proses dimulai dari pembuatan instance virtual menggunakan Multipass, dilanjutkan dengan instalasi Docker, inisialisasi swarm pada node manager, serta penggabungan node worker ke dalam cluster menggunakan token autentikasi. Hasil pengujian menunjukkan bahwa kedua node dapat berkomunikasi dengan baik dan berada dalam status aktif, sehingga cluster siap digunakan untuk menjalankan layanan. Implementasi service menggunakan image nginx berhasil dijalankan dan dapat diakses melalui alamat IP manager maupun worker, yang membuktikan bahwa mekanisme load balancing dan routing internal berjalan dengan optimal. Selain itu, proses pembaruan service menggunakan image yang berbeda dapat dilakukan secara dinamis tanpa menghentikan layanan, serta distribusi task dapat berjalan merata di seluruh node. Dengan demikian, Docker Swarm mampu memberikan kemudahan dalam pengelolaan layanan, meningkatkan ketersediaan sistem, serta mendukung skalabilitas dan fleksibilitas dalam pengembangan infrastruktur berbasis kontainer.